

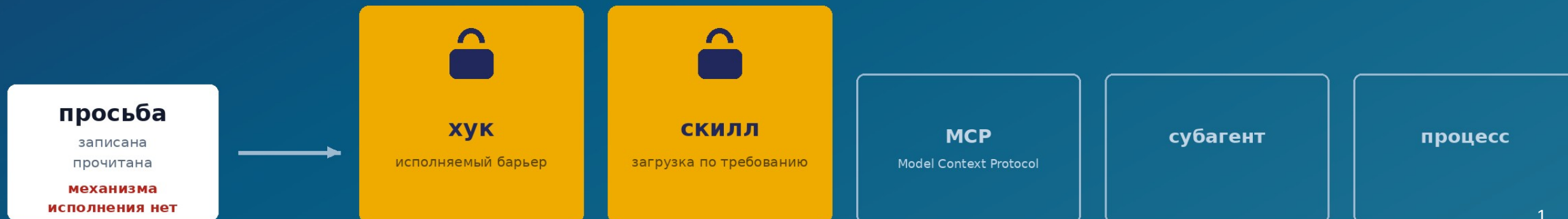
Когда просьбы недостаточно: барьер и загрузка по требованию

«Агент в этом проекте настроен двадцатью четырьмя строками текста. Он их читает. Механизма, который бы их исполнял, в проекте нет.»

«Файл инструкций — это просьба. Что поставить рядом с ней, чтобы исполнение правила не зависело от решения агента?»

сегодня — две из пяти

следующее занятие — три



Где мы остановились

signup-landing — статический лендинг с формой заявки на демо-урок. Готовность определена так: сборка завершается с кодом 0, набор тестов проходит полностью, форма отправлена вручную — в заполненном и в пустом виде.

README.md	spec.md	DECISIONS.md	← что строим и почему
CLAUDE.md	AGENTS.md (ссылка на CLAUDE.md)		← 24 строки, 19 непустых, 4 раздела
doc/adr/	шаблон + 3 ADR + README.md		← как принимались решения
.tasks/	README.md · active/ · done/README.md		← задачная память
index.html	package.json	package-lock.json	
vite.config.js	playwright.config.ts		
src/main.js	src/style.css	src/validate.js	← продукт
tests/form.spec.ts	tests/delivery.spec.ts		
tests/page-objects/form.page.ts			
.gitignore			

.claude/ — нет вовсе: ни хука, ни скилла, ни субагента, ни настроенного сервера

Репозиторий: github.com/tellina-study/signup-landing-demo, ветка `main`.

«Всё перечисленное — текст. Агент может его учесть и может не учесть.»

Пять усилений, которые ставят рядом с файлом инструкций

Рядом с файлом инструкций ставят пять механизмов. По строке на каждый — что это такое, кто приводит его в действие и переносится ли он на другой инструмент.

УСИЛЕНИЕ	ЧТО ЭТО	ПЕРЕНОСИТСЯ ЛИ
Хук	Команда, которую среда запускает сама в заданный момент работы агента.	Приём — у всех. Запись — у каждого своя; у отдельных агентов механизма нет.
Скилл	Процедура в отдельном файле ; агент открывает её сам.	Открытый формат — тот же файл читают десятки агентов.
МСП	Model Context Protocol — подключение к внешней системе; через него агент достаёт данные, которых в репозитории нет.	Открытый протокол — у него нейтральный владелец и общая спецификация.
Субагент	Отдельная роль с урезанными правами ; её запускают на одну задачу и получают обратно только итог.	Приём — у всех. Формат объявления — у каждого свой.
Процесс	Порядок работы, записанный файлами : план, запись по ходу, проверка перед сдачей.	Переносится целиком — это обычные файлы, механизма не требует.

Что бы вы завели первым в этом репозитории

Двадцать четыре строки инструкций, четыре раздела — и ни одного механизма, который бы их исполнял.

В этом репозитории — с его формой заявки, тестами и двадцатью четырьмя строками инструкций — какое из пяти вы завели бы первым? Держите ответ при себе: этот вопрос прозвучит сегодня второй раз.

хук

скилл

МСР

субагент

процесс

Две ступени, шесть развилок

хук	Агент не исполняет процедуру работы с репозиторием решаем чем заставить агента её исполнять	Проверка работала и молча перестала решаем чем её починить	Агент стал работать медленнее из-за хуков решаем что делать с ценой проверок
скилл	Одно правило объявлено в файле дважды, копии разошлись решаем что делать с файлом инструкций	Агент не открывает нужный скилл и открывает ненужный решаем по чему он решает открыть файл	Задача стоит сегодня, а своё решение — это недели решаем брать готовое или писать своё

следующее занятие: MCP · субагент · процесс

К концу занятия дерево репозитория будет отличаться от показанного на втором слайде ровно на две строки.

1 Агент не исполняет процедуру работы с репозиторием. Решаем, чем заставить

Процедура записана в инструкциях и загружена, а коммит всё равно уходит в главную ветку. Решаем, чем заставить агента её исполнять.

1 КЕЙС · 5 РЕШЕНИЙ СБОРКИ · 6 ЗВЕНЬЕВ ОТКАЗА

Хук

Скилл

Запрет написан явно. Коммит всё равно в `main`

«Ветка `case-...` / `issue-NNN-...`, создаётся до первой правки. **Прямо в `main` не коммитим.**»
— файл инструкций проекта, раздел ***Repository etiquette***. Загружается в начале каждой сессии.

- Агент правит проверку формы в `src/validate.js` — задача рабочая, по делу.
- Делает `git commit`. Отдельную ветку не создаёт.
- Коммит лежит в `main`.

Запрет записан. Файл прочитан. Процедура не исполнена. Цена: показ заказчику завтра — правка уезжает мимо ветки прямо в него.

Событие, матчер, решение

БАЗА

Хук — команда, которую **среда запускает сама** в заданный момент работы агента.

Событие — момент запуска. **Матчер** — какие вызовы отбирает. **Решение** — ответ среде.

Устройство целиком — дальше.

событие

матчер

решение

КОГДА ОН ГОВОРИТ И ЧТО ПРИ ЭТОМ ВИДИТ

Момент выбирают при настройке: хук ставят **до** вызова или **после** него.

Хук — отдельная программа. Она знает **только те параметры, которые ей пришли**: какой инструмент, с какими аргументами, в каком каталоге.

Поставленный **до** вызова об исходе не узнаёт: он высказывается один раз, результата ему не присылают. Поставленный **после** видит результат, когда вызов уже состоялся.

Громче написать уже нельзя. Что поставить рядом?

Файл инструкций есть, и инструкция в нём написана настолько громко, насколько это возможно.

В файле инструкций **этого курса**, строка 80: «**NEVER push to main directly ... This is NON-NEGOTIABLE, even for "small" fixes**» — заглавными, со словом «никогда» и отдельной оговоркой про мелочи.

Выберите, что поставить рядом с такой инструкцией — и скажите, почему именно этот ход при четырёх соседних.

защита ветки на сервере

хук

git-хук в .git/hooks

забрать право на git

проверка руками

Пять ходов, у каждого свой момент и своя цена

ВАРИАНТ	ГДЕ ОН РАБОТАЕТ — КОНКРЕТНЫЕ СЛУЧАИ	ОГРАНИЧЕНИЯ
защита главной ветки на стороне сервера	правило держится независимо от того, кто и чем работает: агент, человек, другая машина, чужой клон	узнаёт о нарушении после отправки: коммит уже в локальной истории. Настраивается вне репозитория и другим человеком, с клоном не приезжает
хук	до выполнения команды (или после) нужна проверка, которая её разрешает, запрещает или отправляет к человеку	видит только агента: та же команда, набранная человеком в оболочке, идёт мимо. Проверяет форму команды, а форм у одной команды несколько
git-хук в <code>.git/hooks</code>	команды набирает человек, и проверка нужна на самом git — одна для всех способов работы	<code>.git/hooks</code> не версионизируется: в каждый клон копируется руками. В репозитории этого курса такой хук лежит и не работает
забрать у агента право запускать git	агент не должен трогать историю вообще: пишет код, коммитит человек	в этом проекте агент коммитит по делу всю сессию — запрет на инструмент остановит работу вместе с нарушением
проверять руками перед показом	показы редкие, и за каждым стоит человек, который помнит про ветку	срабатывает после коммита и держится на памяти человека

Из сочетания колонок: первые две строки законны обе. На стороне сервера правило держится крепче — его не обойти формой команды. Хук держится раньше — он успевает до появления коммита. Разбираем агентов и момент, в который действует агент, поэтому собираем хук; серверную защиту ставят поверх, вторым слоем.

Хук целиком, одним экраном

1 ГДЕ ОБЪЯВЛЯЕТСЯ

<code>.claude/settings.json</code>	— файл в git, едет с клоном
<code>PreToolUse</code>	— событие: до вызова инструмента
<code>matcher</code>	— на какие вызовы реагировать
<code>command</code>	— что запустить
<code>timeout</code>	— сколько ждать ответа

3 ЧТО ПРИХОДИТ НА ВХОД — описание вызова

<code>{ "tool_name": "Bash",</code>	какой инструмент
<code> "tool_input": {"command": "git commit ..."},</code>	его аргументы
<code> "cwd": "/.../signup-landing",</code>	рабочий каталог
<code> "permission_mode": "default" }</code>	режим прав

4 ЧЕМ ОТВЕЧАЕТ — два канала, и работают они по-разному

код возврата

0 — решение напечатано

1 — хук сломался, вызов идёт

2 — блокировать

объект решения

запретить

спросить

разрешить

поля нет —
обычный порядок

5 ЧТО СРЕДА ДЕЛАЕТ С ОТВЕТОМ

модель собрала
аргументы



сработало
событие



хук получил
описание вызова



хук напечатал
решение



среда определила:
вызов осуществляется или пропускается

Запрет снимает только тот вызов, на который сработал хук. Агент продолжает работу и читает причину отказа.

2 КОГДА ЗАПУСКАЕТСЯ — одно событие из тридцати трёх

вызов инструмента

6 событий

файлы и каталоги
5 событий

сессия и настройки

5 событий

команда агентов
5 событий

ход пользователя

4 события

контекст и среда
8 событий

Корзина — группа событий. Нам нужна одна: вызов инструмента

6 САМ КОД — ГДЕ ЛЕЖИТ И НА ЧЁМ ПИШЕТСЯ

<code>"command": "jq -r '.tool_input.command' ..."</code>	в поле настроек
<code>"command": ".claude/hooks/guard-main.sh"</code>	или файл рядом

Запускается как программа — оболочка, Python, что угодно.

Вызов читает со стандартного входа, решение печатает на выход.

Проектируем барьер: на что, когда и как реагировать

ЧТО РЕШАЕМ	ВАРИАНТЫ	ЧТО МЕНЯЕТ ВЫБОР
на что реагировать — матчер	любой инструмент · определённый инструмент · команды оболочки · только правки файлов	чем шире матчер, тем чаще запускается хук и тем дороже любая ошибка внутри самой проверки
когда реагировать — событие	до вызова · после вызова	после вызова остаётся запись факта: коммит уже в истории
как реагировать — что считать нарушением	любая команда <code>git</code> · только коммит · только коммит в защищённой ветке	чем точнее условие, тем меньше ложных отказов и тем больше форм команды проходит мимо
как реагировать — чем отвечать	запретить · спросить человека · разрешить и записать	запрет снимает один вызов, работу агента он не останавливает; «спросить» передаёт решение человеку и требует человека на месте
как реагировать — если проверка не смогла ответить	разрешить молча · запретить · спросить человека	проверка, которая не смогла ответить и промолчала, снаружи выглядит пройденной

Собранный барьер: девять строк, у каждой своя работа

БЛОК КАРТЫ	ЧТО СТОИТ В ФАЙЛЕ	ЧТО ЭТА СТРОКА ДЕЛАЕТ
1 объявление	"PreToolUse"	событие: среда запускает проверку до того, как вызов исполнится
1 объявление	"matcher": "Bash"	отбирает только вызовы команд оболочки; правки файлов идут мимо
6 код	шаг 1 · jq -r ' .tool_input.command '	достаёт из описания вызова (блок 3) саму команду одной строкой
6 код	шаг 2 · grep -qE по этой строке	ищет в ней git commit — в начале строки или после разделителя команд
6 код	шаг 3 · git rev-parse --is-inside-work-tree	спрашивает, репозиторий ли это вообще
4 ответ	"permissionDecision": "ask"	репозитория нет: решение передаётся человеку, молча не пропускаем
6 код	шаг 4 · git symbolic-ref --short HEAD	спрашивает имя текущей ветки; отвечает и на репозитории без коммитов
4 ответ	"permissionDecision": "deny"	ветка main или master: вызов закрыт, причина уходит агенту
1 объявление	"timeout": 10	сколько секунд среда ждёт ответа, прежде чем отменить проверку

Барьер стоит. Срабатывает не всегда

Мини-кейс. Барьер стоит, самотест проходит, а в главной ветке снова чужой коммит. Сообщения об ошибке нет нигде.

ЗВЕНО	ЧТО ПОРВАЛОСЬ	ЧЕМ ПРОВЕРЯЕТСЯ
1 · объявлен среде	записи в файле настроек нет, среда про хук не знает	<code>/hooks</code> — список настроенных хуков. Нашего в нём нет
2 · запись принята	сломанный JSON или <code>matcher</code> массивом: среда отклонила	<code>claude doctor</code> — называет отклонённое и причину
3 · матчер совпал	другой регистр или опечатка в имени инструмента	<code>claude --debug-file</code> — в журнале видно, что сверялось
4 · скрипт запустился	файла нет, нет права на исполнение, нет <code>jq</code>	прогнать руками: <code>bash guard-main.sh; echo \$?</code> даёт код возврата
5 · успел ответить	вышел <code>"timeout"</code> : хук отменён, вывод отброшен, вызов идёт	<code>time bash guard-main.sh</code> — сколько проверка занимает
6 · ответ прочитан	поле решения не на своём уровне либо вывод не JSON	тот же журнал: вывод хука не начинается с фигурной скобки

Проверять сверху вниз: первое порванное звено объясняет всё, что ниже. Седьмой случай — не поломка: цепочка цела, проверка сработала и пропустила, потому что ищет форму команды. Это предел, он на следующем слайде.

До какого момента он честно работает

Барьер ставим, список известных дыр записываем и держим на виду. Регулярным выражением его не закрываем: ниже — три предела и то, чем их закрывают.

ЭТОТ ХУК ЧЕСТНО РАБОТАЕТ, ПОКА

правило выражается **формой команды**

нарушитель — **агент**

форм команды **конечное число**

ЗА ПРЕДЕЛОМ

правило-суждение формой не выражается: «не делай рискованных правок перед показом» механизировать нечем. Остаётся текст плюс проверка человеком

вы сами, руками, в терминале закоммитите в главную — этого события хук не увидит: оно принадлежит жизни агента. Нужен git-хук или запрет на стороне сервера

их число не конечно: оболочка раскрывает слова **после** решения хука. `env git commit, /usr/bin/git commit`, коммит на второй строке составного вызова — проверка на строку их не ловит. Потолком ставят запрет на стороне платформы, а хук сужают до **перечислимого**: `--no-verify`, подмена `core.hooksPath`

2

Решаем, что делать, когда проверка работала и молча перестала

Проверка стояла, срабатывала и молча перестала — а файл её никто не трогал.

1 ИНЦИДЕНТ · 5 РЕШЕНИЙ · 2 ХУКА, КОТОРЫЕ ПЕРЕСЕКЛИСЬ

Хук

Скилл

Проверка перестала срабатывать

ЧТО ВИДЕЛИ · ПО ПОРЯДКУ

ЧТО ВИДНО

две недели назад поставили

проверку на правки защищённых файлов. Прогнали на живой правке — остановила, причину напечатала

сегодня заметили

в защищённом файле новые строки. Остановить их было некому: подтверждения никто не давал и отказа никто не видел

пошли смотреть

запись о проверке на месте, файл с того дня не открывали. Ни ошибки, ни предупреждения — ни в сессии, ни в журнале

Что могло измениться, если сам хук не трогали? Назовите две-три версии.

Изменили права. Агент сменил инструмент

ЧТО ОКАЗАЛОСЬ

ЧТО ИМЕННО ПРОИЗОШЛО

1 · изменили права

кому — агенту в этом проекте. **На что** — запускать команды оболочки без подтверждения. **Где** — в режиме запуска сессии; файл хука при этом не открывали. **Как** — одним переключением режима, сразу на всю сессию

2 · агент сменил инструмент

в этом режиме среда добавляет агенту указание: предпочитать команды оболочки отдельным инструментам правки. Агент перестал звать правку файла и стал делать ту же правку командой `sed -i` — она подменяет строку прямо в файле, на месте

3 · матчер отбирает по имени

в матчере перечислены `Edit` и `Write` — правка файла и запись файла. Команды оболочки в перечне нет. Хук не сломался: вызов до него не дошёл

И это не у всех: указание активно для одной модели, по умолчанию не активно для двух других. У соседа с той же настройкой проверка работает — поэтому «у меня же всё срабатывало» здесь ничего не доказывает.

Что делаете?

Проверка стоит, написана правильно и однажды работала. Матчер перечисляет имена инструментов; правка пришла командой оболочки и прошла мимо.

Назовите ход — и скажите, что он вам даст.

расширить матчер

переписать логику
хука

поправить
инструкции агенту

поставить второй хук

вернуть прежний
режим

Пять решений, один матчер

РЕШЕНИЕ	ГДЕ ОНО РАБОТАЕТ — КОНКРЕТНЫЕ СЛУЧАИ	ОГРАНИЧЕНИЯ
расширить матчер	запрет один и тот же, а путей к действию несколько, и все они известны и перечислимы: правка файла, запись файла, команда оболочки. Починка в одну строку настроек, скрипт при этом не трогаем	перечень закрыт: путь, которого в нём нет, проходит молча. И широкий перечень запускает проверку на каждом вызове оболочки, а их за задачу десятки — цену этого считаем в следующем случае
переписать логику хука	путей к действию больше, чем вы готовы перечислять, а признак у них общий: трогают защищённый файл. Проверяем предмет правки, и тогда путь перестаёт иметь значение	предмет надо достать. У правки файла имя лежит готовым полем; у команды оболочки оно внутри строки, и форм у строки несколько — та сложность, с которой мы встретились в первом случае
поправить инструкции агенту	правило мягкое, цена нарушения мала. И как дополнение к барьеру: агент знает, чего от него хотят, и причина отказа ложится на подготовленное место	инструкция остаётся текстом. Что бывает с текстом, мы разобрали час назад: правило было написано громче некуда, и агент прошёл мимо
поставить второй хук	проверки разные по существу: одна читает готовые аргументы правки, вторая разбирает строку команды. И когда первый хук работает, а трогать его не хочется	на одном событии срабатывают оба и отвечают независимо. Их ответы сводит среда, по своему старшинству, — его придётся знать
вернуть прежний режим	широкие права включали для удобства, требованием они не были. И когда закрыть надо сегодня, а перебирать все пути правки некогда	закрывает один известный способ; класс остаётся открытым. Режим включают снова — повторится то же самое; у соседа на другой модели этого указания и так нет

Из сочетания колонок: расширить матчер и поставить второй хук законны оба — разница в том, одна у вас проверка или две. Проверка здесь одна и та же: тот же файл, тот же запрет. Поэтому расширяем матчер, а логику правим следом — имя инструмента отбирает вызовы, решает предмет правки. Инструкции дописывают поверх, возврат режима остаётся мерой на сегодня.

Четыре способа не совпасть

В первом случае мы разобрали цепочку из шести звеньев, и каждое рвётся молча. Здесь разворачивается одно из них — то, где сверяется матчер. Четыре способа не совпасть; остальные пять звеньев остаются там, где были.

ЧТО С МАТЧЕРОМ

ПОЧЕМУ НЕ СОВПАЛО

ЧЕМ ПРОВЕРИТЬ

имя инструмента написано с опечаткой или в другом регистре

сверяются строки, и `bash` против `Bash` — разные строки. Несовпадение ошибкой не считается

`/hooks` — встроенный список настроенных проверок. Наша в нём есть и не запускается ни разу: это и есть признак

имя правильное, но агент вызвал другой инструмент

наш случай: та же правка пришла командой оболочки

`claude --debug-file <файл>` — пишет подробный журнал запуска в указанный файл; видно, какой инструмент вызывался и с чем сверялся матчер

перечень имён задан списком вместо строки

это ошибка схемы: среда отклоняет файл настроек целиком, и ни одна проверка из него не работает

`claude doctor` — запускается вне сессии, разбирает файлы настроек и называет отклонённый файл и причину

перечень расширили, и он стал совпадать на всём

проверка запускается на каждом вызове оболочки, в том числе там, где решать нечего

`time bash <скрипт>` — сколько занимает один прогон. Умножьте на число совпадений за задачу

Решение вы выбрали — его ещё надо увидеть сработавшим. Матчер, который ничего не отбирает, и матчер, которому нечего отбирать, снаружи выглядят одинаково: тишиной.

Сработали оба. Чей ответ получит агент

Первый хук вы не трогали — поставили рядом второй, со своим условием на команды оболочки. На правке через `sed -i` совпали оба.

ПОСЛЕДОВАТЕЛЬНОСТЬ ПРИМЕНЕНИЯ ↓	ЧТО ДЕЛАЕТ СРЕДА	ЧТО ИЗ ЭТОГО СЛЕДУЕТ
1	запускает все проверки, чьё условие совпало	запускаются обе, одновременно. Друг о друге они не знают и ответы не согласуют
2	читает коды возврата	код 2 у любой из двух блокирует вызов — даже против «разрешить», напечатанного ею же
3	сводит решения по старшинству: deny → defer → ask → allow	запрет одной перебивает разрешение другой. Запрет сильнее разрешения
4	смотрит, дала ли решение хоть одна	не дала ни одна — решения нет вовсе, среда идёт обычным порядком разрешений. Это не «разрешено»

Матчер расширили — вызов всё равно прошёл

Решение вы приняли верно: перечень теперь ловит и команды оболочки. Вот три способа, которыми именно расширенная проверка отдаёт вызов дальше.

ЧТО СЛУЧИЛОСЬ	ВЫ ПРОСИЛИ	СРЕДА СДЕЛАЛА
проверка не ответила за отведённый срок — а запускается она теперь на каждом вызове оболочки, и срок выходит чаще прежнего	запретить	пропустила вызов
j q в системе нет: скрипт умер на первой строке и вернул код 1, а код 1 среда считает неблокирующей ошибкой	запретить	пропустила вызов
поле решения лежит снаружи hookSpecificOutput; внутри, где среда его читает, пусто	запретить	пропустила вызов

Во всех трёх в записи сессии — ровно то же самое, что при исправной работе. Расширенный перечень лежит в файле, вы его видите. Действием он быть перестал.

До какого момента расширенного перечня достаточно

Решение: перечень расширяют один раз, доказательство его срабатывания обновляют. Граница проходит по доказательству; длина перечня её не задаёт.

РЕШЕНИЯ ДОСТАТОЧНО, ПОКА

доказано, что проверка срабатывает на новом пути — её заставили отказать громко хотя бы раз именно на команде оболочки, и это повторяют после каждого обновления среды и каждой смены прав

путей к действию столько, сколько вы перечислили — перечень закрыт и вы готовы его пополнять

ЗА ПРЕДЕЛОМ

пока доказательства нет, «запретить» и «пропустить» снаружи неразличимы: и то и другое выглядит тишиной

путь, которого в перечне нет, проходит молча. Перечень имён перестаёт быть опорой: проверяют предмет — какой файл правка собирается менять

3

Решаем, что делать, когда агент стал работать медленнее из-за хуков

Барьеры стоят, отрабатывают исправно и не выдают ни одной ошибки — а работа идёт в разы дольше, чем шла месяц назад.

1 СЛУЧАЙ · 3 ЗАМЕРА · 5 ВАРИАНТОВ ДЕЙСТВИЯ

Хук

Скилл

Та же задача: было — минуты, стало — час

Одна линейка на оба состояния: минуты и час

было

до пересмотра

стало

через несколько недель



ЧТО ВИДНО ПО ЧАСАМ

ЗНАЧЕНИЕ

до пересмотра процессов в репозитории

задача такого размера занимала **минуты**

через несколько недель после пересмотра

та же задача занимает **час**

что изменилось в самой задаче

ничего: тот же проект, тот же размер правки, та же машина

Что случилось? Назовите две-три версии.

В журналах: десятки запусков хука, каждый прогон долгий

ЧТО ВИДНО В ЖУРНАЛАХ

запись о запуске хука идёт почти **на каждом шагу агента** — за задачу их **десятки**

внутри запуска проходит **полный цикл проверок**, и сам цикл не быстрый

ЧЕМ ЭТО ВЫХОДИТ ПО ЧАСАМ

каждый запуск — **пауза**: агент стоит и ждёт

десятки долгих прогонов и дают тот час

Ошибок в журнале нет ни одной. Хук отрабатывает штатно — просто очень часто, и каждый прогон долгий.

Что делаете — и с чего начинаете?

Та же задача идёт час. Хук вызывается много раз за задачу, каждый вызов долгий, ошибок нет ни одной.

Выберите, что делаете — и с чего начинаете.

**терпеть — это цена
безопасности**

выкинуть хуки

увеличить таймаут

сузить матчер

**вынести дорогие
проверки на пул-
реквест**

Пять ходов: что каждый меняет в цене задачи

ВАРИАНТ	ГДЕ ОН РАБОТАЕТ — КОНКРЕТНЫЕ СЛУЧАИ	ОГРАНИЧЕНИЯ
терпеть — это цена безопасности	задержка короче, чем разбирательство с ней, и проверка стоит на одном действии за задачу: барьер ветки добавляет миллисекунды и спрашивать тут не о чем	«цена безопасности» названа, но не посчитана. Пока число не известно, терпят догадку о цене. И потеря невидима: сообщения нет, поэтому замечают её позже, чем начинают платить
выкинуть хуки	измерение показало, что проверка дорогая, а её работу делает другой барьер — защита на стороне сервера или проверка на пул-реквесте	снимает вместе с дорогим и то, что стоило миллисекунды и работало. Возвращает картину первого случая: правило записано, исполнять его некому
увеличить таймаут	проверка иногда обрывается на полпути, а её ответ нужен целиком: тогда срок поднимают, чтобы она доходила до конца	таймаут задаёт, сколько среда ждёт ответа. Когда срок вышел, вызов идёт дальше без проверки, и наружу об этом не печатается ничего. Увеличенный срок удлиняет ожидание и оставляет тот же тихий пропуск; скорость самой проверки он не меняет
сузить матчер	проверка нужна на одном действии из многих, и это действие опознаётся по вызову: матчер перестаёт запускать её на остальных	сужение надо обосновать замером, иначе из-под проверки уедет нужное. И слишком узкий матчер пропускает путь, которого в нём нет, — ровно второй случай
вынести дорогие проверки на пул-реквест	сборка, тесты, разбор кода: их ответ нужен один раз, когда работа готова	ответ приходит в конце задачи, когда правка давно сделана. Механизм живёт вне репозитория, настраивается и чинится отдельно

Из сочетания колонок: цена задачи — это число срабатываний, умноженное на цену одного прогона. Срок ожидания произведение оставляет прежним. Отказ от проверок обнуляет его вместе с тем, что работало. Остаются два хода, и порядок между ними задан величинами: сначала сужают матчер — число прогонов падает в разы одной строкой настроек; затем дорогие проверки уводят туда, где они идут один раз за задачу. Терпеть остаётся законным ходом — после того, как цену посчитали.

Решение принято — вот откуда берётся час



Как померить у себя

СРЕДСТВО	ЧТО ОНО ДЕЛАЕТ И ЧТО ДАЁТ
/hooks	показывает, какие хуки стоят и с какими матчерами — список целиком, вместе с теми, что приехали с настройками. Начинать надо отсюда
попросить агента посмотреть журнал	самое дешёвое и самое забываемое: у агента есть доступ к тем же файлам, он прочитает отладочный журнал и назовёт, какие хуки срабатывали и сколько заняли
прогон команды хука под time × число правок за типичную задачу	time измеряет, сколько заняло одно срабатывание — от запуска до ответа. Умноженное на число правок, это цифра за задачу: единственная, которая отвечает на вопрос
профилировщик хуков claudekit	то же измерение, снятое за вас на всех хуках сразу — готовый инструмент стороннего автора
запуск с записью отладочного файла	показывает каждый запуск построчно: сработал ли хук, на какой вызов и что вернул

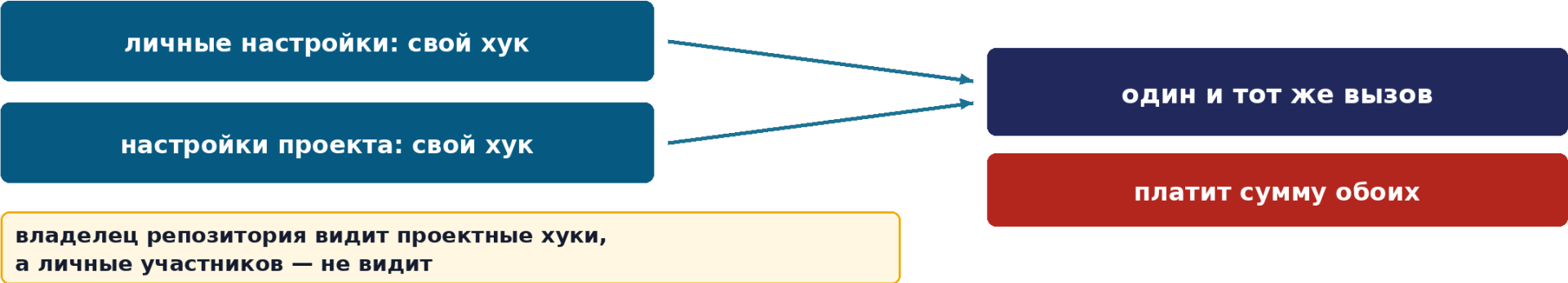
Процедура, три шага: ① посчитать правки за задачу → ② умножить на цену одного прогона → ③ сравнить с длительностью задачи.

Порог — это доля задачи. Измеренная цена правки — 1 275 мс (сборка 1 137 мс плюс синтаксическая проверка 124 мс), сорок правок дают 51 с:

В ЗАДАЧЕ НА	51 С СОСТАВЛЯЮТ
10 минут	8 % — терпимо
3 минуты	28 % — почти треть
1 минуту	85 % — почти столько же, сколько сама задача

Вы измерили свой хук — сессия платит не за него

Два уровня настроек — и оба попадают в один вызов



Списки хуков не перекрываются — они объединяются. Личный хук из ваших домашних настроек и проектный хук на то же событие сработают оба.

ВЫ ИЗМЕРИЛИ	СВОЙ ХУК
сессия платит за	сумму всех сработавших
владелец репозитория видит	только проектные — те, что каждый участник завёл себе сам, ему не видны

Честный пробел: сколько хуков в среднем стоит в реальном проекте и какова суммарная латентность по флоту — не измерено никем. Контролируемого эксперимента с единым хуком на едином наборе вызовов не существует.
Поэтому забирайте отсюда **арифметику и способ считать**: готового числа здесь нет.

Что проверять на каждом шаге, а что — один раз

Правило держится на частоте. Проверка, которая срабатывает на каждом шаге, умножается на число шагов. Растёт частота; цена одного прогона остаётся прежней. Поэтому проверки сортируют.

За пределом правила: проверка, которую нельзя отложить и которая всё равно стоит секунды. Сортировкой её не спасти — меняют саму проверку: сокращают до того, что нужно прямо сейчас, а полную оставляют на конец работы.

ПРОВЕРЯТЬ НА КАЖДОМ ШАГЕ

то, без чего следующий шаг опасен — запрет должен сработать в ту секунду, когда действие происходит, иначе он бесполезен

то, что стоит миллисекунды — такую задержку человек не замечает

наш барьер: **27 мс, один раз за работу** — он смотрит на одно действие и на каждое изменение файла не реагирует

ПРОВЕРЯТЬ ОДИН РАЗ ЗА РАБОТУ

то, что подтверждает результат — сборка, тесты, разбор кода: их ответ нужен в конце работы

то, что стоит секунды — секунда на шаге при двадцати шагах превращается в полминуты ожидания

сборка проекта: **1 137 мс на каждой правке**; двадцать правок — около 23 секунд чистого ожидания

- 23 секунды — арифметика: $1\,137 \text{ мс сборки} \times 20 \text{ правок}$, **только сборка**. Цены сняты на демо-репозитории, по 100 прогонов на каждый хук.
- Барьер укреплял то, что уже лежит в файлах проекта. Дальше — про то, чего в файле лежать не должно.

4

Решаем, что делать, когда одно правило объявлено в файле дважды и копии разошлись

Одно правило объявлено в файле инструкций дважды, копии разошлись — и агент читает обе в каждой сессии, включая те, где это правило ни при чём.

2 КОПИИ ОДНОГО ПРАВИЛА · 4 ЗАМЕРА · 1 ПРОВАЛ

Хук

Скилл

Файл, который спорит сам с собой

Что это за файл. `CLAUDE.md` — постоянная инструкция агенту в этом курсе. Агент читает её целиком, при старте каждой сессии, — в том числе в той, где его просят поправить отступ в вёрстке. Команда ниже ищет строку по всему файлу и печатает номера строк, где она встретилась.

ЧТО НАШЛОСЬ

правило «какой длины обязана быть глава методички» объявлено **дважды**, и копии расходятся: одна про разбиение длинной главы, другая добавляет, что засчитывается в объём, и отменяет прежний ориентир

правило, которое само пишет, что **перекрывает соседнее**

длина файла — **690 строк**

БАЗА СРАВНЕНИЯ

строки **117** и **474** одного файла

строка **216**; оговорку дописали руками — после того как конфликт сработал на живой лекции

собственный предел курса, **600**, объявлен в этом же файле

```
$ grep -n "Chapter Depth Baseline (ENFORCED" CLAUDE.md
117:## Chapter Depth Baseline (ENFORCED – фундаментальное)
474:## Chapter Depth Baseline (ENFORCED – фундаментальное, issue #128)
```

Никто не приходил и не говорил, что сломалось: заметно ничего и не ломалось. Обе копии написаны в разные дни разными заходами и нашлись при подготовке этого слайда.

Три уровня загрузки — и цена каждого

БАЗА

Скилл — процедура в **отдельном файле**; агент открывает её сам.

Три уровня: имя и описание (~100 токенов), тело (до ~5 тыс.), вложения (ноль).

имя и описание

тело файла

вложения

ЗА ЧТО ПЛАТЯТ ВСЕГДА

Первый уровень платится **на каждом ходу** — **вызовется скилл или нет**.

Третий не платится вовсе, пока не открыт, и практического предела по объёму у него нет.

Дословно из документации: перечень скиллов прибавляется к контексту на каждом ходу — вызывал их агент хоть раз или нет.

Что делаете с этим файлом?

Файл перед вами. Одно правило объявлено дважды, копии разошлись, и агент читает обе в каждой сессии — включая ту, где его просят поправить отступ в вёрстке. Файл при этом на 690 строк при собственном пределе в 600.

Выберите, что делаете — и скажите, почему именно этот ход при четырёх соседних.

свести копии в одну

вынести в скилл

дописать, какая копия
главная

сократить файл до
600 строк

оставить как есть

Пять действий, у каждого свой случай и своя цена

ВАРИАНТ	ГДЕ ОН РАБОТАЕТ — КОНКРЕТНЫЕ СЛУЧАИ	ОГРАНИЧЕНИЯ
свести копии в одну	правило нужно почти в каждой сессии и должно остаться в файле инструкций; копий две-три и они про одно и то же; расхождение надо убрать сегодня, до следующего запуска	не отвечает, где этой процедуре жить дальше: через три месяца рядом напишут третью версию, и всё начнётся заново. Именно так и получилась вторая
вынести в скилл	тело большое, а нужно редко — только когда пишут главу; две версии правила спорят, и вынос разводит их по разным загрузкам; процедура исполняемая — скилл приносит её скриптом и возвращает готовый вывод	имя и описание платятся в каждом ходу, вызовется скилл или нет. И вынос работает только вместе с зачисткой: копия, забытая в файле инструкций, оплачивается вторично
дописать, какая копия главная	конфликт один, известен, и развести копии прямо сейчас дороже, чем оговорить приоритет; правило чужое, и в этом заходе его не правят	работает ровно один раз — именно это и сделано на строке 216 этого файла . Следующую пару придётся оговаривать снова, и оговорок станет столько же, сколько конфликтов
сократить файл до 600 строк	файл читает человек, и шестьсот строк — предел того, что он удержит в одном документе. Как мерка читаемости она верна и поставлена сознательно	на соблюдение не влияет, и это проверено напрямую: ни размер, ни положение правила, ни устройство файла. При двухстах строках тот же спор стоил бы столько же
оставить как есть	спорный раздел в этом проекте не запускают: правило про то, чего здесь не делают	«оно не ломается» никто не проверял. Цена столкновения двух правил измерена. Обе копии при этом продолжают грузиться в каждую сессию

Из сочетания колонок: первые две строки законны обе, и порядок между ними имеет значение. Сведение убирает спор сегодня и молчит о том, где правилу жить. Вынос убирает повторение спора — но сводить всё равно придётся, иначе вынесется одна из двух правд. Поэтому сначала одна правда, потом её место: свести, затем вынести. И вынос считается сделанным только после зачистки — без неё за одну процедуру платят дважды.

Чем подтверждается каждый шаг решения

ЧТО ИЗМЕРЕНО

при столкновении двух правил соблюдаются **оба сразу** — в **35,4%** случаев

устройство файла — размер, положение правила, архитектура — на соблюдение **не влияет** статистически

чем дольше идёт сессия, тем хуже соблюдается правило: **–5,6% шансов на каждой следующей написанной функции**

вложения не стоят ничего, пока не открыты, и предела по объёму не имеют

ОТ КАКОЙ БАЗЫ

13 335 испытаний на 6 публичных наборах системных правил; мерилось соблюдение обоих правил пары одновременно

1650 сессий, 16 050 наблюдений, контролируемый эксперимент на четырёх характеристиках файла

те же 1650 сессий; считали так: агент пишет функцию за функцией, и каждую следующую сравнивали с предыдущей в той же сессии

официальная документация; ограничение там по глубине ссылок — один уровень, и размера оно не касается

ЧТО ЭТО ПОДТВЕРЖДАЕТ

цена спора, который мы только что развели: пока копии стоят рядом, в двух случаях из трёх одна из них не соблюдена

ставку на сокращение замер не поддерживает ничем: мерка читаемости верна сама по себе и к соблюдению отношения не имеет

почему вынос сильнее сведения в долгую: то, что лежало в контексте с самого начала, к концу сессии работает хуже всего

крупное выносить не страшно: объём платить не заставит, пока его не открыли

Оговорка про обратную величину. 64,6% — это доля тех же 13 335 испытаний, где нарушено хотя бы одно правило пары, и столкновения в них сконструированы специально. Это не оценка того, как часто правила нарушаются сами по себе; авторы предупреждают об этом первыми.

Скилл целиком, одним экраном

1 где лежит и из чего состоит

```
.claude/skills/deploy/  
SKILL.md  
---  
name: deploy  
description: Деплоит лендинг...  
---  
# deploy  
1. Собрать: npm run build  
checklist.md  
check.sh
```

имя каталога = имя скилла
единственный обязательный файл

ФРОНТМАТТЕР

ОПИСАНИЕ пишут третьим лицом

ТЕЛО

ВЛОЖЕНИЕ читается по ссылке из тела
запускается, в контекст идёт вывод

3 чем вызывается

модель выбирает сама — по описанию

человек зовёт по имени — /deploy

/deploy — открыть скилл с этим именем сразу, минуя выбор по описанию
ручной вызов работает даже когда описание никуда не годится —
отсюда целый класс поломок, которых автор у себя не замечает

5 путь от запроса до исполнения



2 что попадает в контекст и когда

имя и описание

когда: всегда, с первого хода сессии
кто решает: никто — попадает само

тело файла

когда: когда скилл выбран
кто решает: модель, по описанию

вложения

когда: когда тело на них сослалось
кто решает: тело файла

4 перечень — всё, что видно до выбора

build-deck: build-deck
pre-user-gate: Pre-USER-GATE walkthrough – orchestrator...
...по строке на каждый установленный скилл

ПЕРЕЧЕНЬ ни тела, ни файлов внутри до выбора не видно

Вынесли правильно, зачистить забыли

ГДЕ ЛЕЖИТ	КОГДА ГРУЗИТСЯ	ДЛИНА
<code>.claude/skills/pre-user-gate/SKILL.md</code> — вынесенная процедура	когда агент выбрал её по описанию	165 строк
<code>CLAUDE.md</code> , строки 252–279 — её копия	при старте каждой сессии, без исключений	28 строк

```
.claude/skills/pre-user-gate/SKILL.md
  1. Visual sweep: открыть все снимки слайдов ...
  2. Notes read: 5-7 случайных заметок ...
CLAUDE.md
## Pre-USER-GATE Walkthrough Rule (ENFORCED)
  1. Mandatory: invoke /pre-user-gate skill ← отправляет обратно в скилл
```

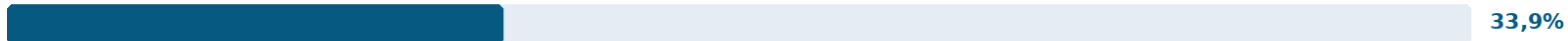
Копия существует затем, чтобы первым же пунктом отправить читателя в файл, который лежит рядом. Вынос сделан 17 мая; зачистка не сделана до сих пор — четыре с половиной месяца за одну процедуру платят дважды.

Плюс шестнадцать пунктов — только у отобранного

Прирост измерен на скиллах, которые кто-то прочитал и отобрал

успешных прохождений: те же 87 задач, те же 18 связок модели со средой

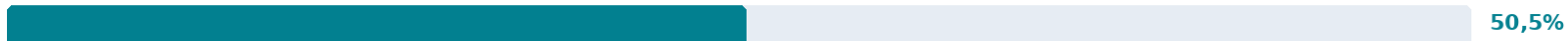
без скиллов



33,9%

+16,6 п.п.

с курированными



50,5%

ЧТО ИЗМЕРЕНО

33,9% → 50,5% успешных прохождений, то есть **+16,6 процентных пункта**

БАЗА СРАВНЕНИЯ

те же 87 задач и те же 18 связок «модель плюс среда»; единственное, что менялось — наличие **курированного** скилла

Плюс шестнадцать пунктов — это про скилл, который кто-то отобрал. Замер поставлен на отобранных: он говорит, чего стоит хороший скилл, и молчит про средний — про тот, что вы взяли с полки.

Когда скилл, когда строка, когда вложение

СКИЛЛ ЧЕСТНО ОКУПАЕТСЯ

тело большое, а срабатывает редко: платите ~100 токенов постоянно, экономите тело в тех сессиях, где оно не нужно

содержимое читается целиком, когда понадобилось: одна процедура, один заход

скиллов в проекте единицы, и области у них не пересекаются

ЗА ПРЕДЕЛОМ — И ЧТО ВМЕСТО НЕГО

процедура нужна **каждую** сессию → строка в файле инструкций: тело и так будет грузиться почти всегда, а шаг угадывания добавится

объём огромен, а нужны из него куски → **вложение**: ноль, пока не открыто, и практического предела по объёму нет

их десятки → выигрыш съедает постоянная плата за первый уровень — те самые ~100 токенов на каждый скилл в каждом ходу — плюс растущая вероятность выбрать не тот

- Возьмите процедуру из своей работы и скажите, в какую строку она попала. Если ни в какую — назовите её, разберём вслух.
- Повод есть, и он не про длину файла. Дальше всё упирается в одну строку — ту, по которой модель решает, открывать файл или нет.

5

Решаем, что делать, когда агент не открывает нужный скилл и открывает ненужный

Тело скилла в контексте не лежит. До срабатывания модель видит одну строку — и по ней решает, открывать файл или нет.

3 ЧАСТИ ОПИСАНИЯ · 2 ДЛИНЫ, КОТОРЫЕ ПУТАЮТ

Хук

Скилл

build-deck: build-deck

перечень скиллов этой сессии, как его видит модель:

```
build-deck: build-deck
catalog-docs: catalog-docs
compile-wiki: compile-wiki
diagram-refresh: diagram-refresh
    ... ещё семь строк ровно такого же вида ...
pre-user-gate: Pre-USER-GATE walkthrough — orchestrator self-review
    before presenting GATE to user.
update-lecture: update-lecture
```

```
$ head -3 .claude/skills/build-deck/SKILL.md
# build-deck
```

Build/rebuild a PowerPoint presentation for a lecture using the
repo-first pipeline + visual loop + 3 QA agents.

Тринадцать строк, из них двенадцать — имя вместо описания. Почему: фронтматтера в файле нет, и описанием становится его первая непустая строка — заголовок, который дублирует имя. Годный текст лежит третьей строкой того же файла и в перечень не попадает. Проверяется одной командой: `head -5 .claude/skills/*/SKILL.md` — она печатает первые пять строк каждого файла скилла, и по ним сразу видно, начинается файл с фронтматтера или сразу с заголовка.

Описание идёт в промпт — и его могут урезать

БАЗА

В системный промпт агента уходят имя и описание — по ним и делается выбор. Тела файла до срабатывания там нет.

описание

бюджет перечня

урезание

ЧТО ПРОИСХОДИТ, КОГДА ПЕРЕЧЕНЬ НЕ ПОМЕЩАЕТСЯ

У перечня есть бюджет — **1% окна модели**. Когда он переполнен, среда **отнимает описания, начиная с тех скиллов, которые вызывают реже всего**: сначала теряет текст тот, кто и так ни разу не сработал.

Две длины, и они про разное. Само поле описания — **1024 знака**. Одна строка перечня — **1536**, и при переполнении у неё обрезается **конец**. Отсюда правило, которое стоит унести: **ключевой случай — первым**, потому что теряете вы хвост.

Нашей деке урезание не грозит: тринадцать скиллов — около 1 300 токенов против примерно 2 000 бюджета при окне в 200 тысяч. Порог от нас далёк, знать надо механизм.

Какое описание сработает?

Пишем описание тому самому *build-deck*, который сейчас виден в перечне как имя-дубль. Проверок к нему две, и обе обязательны: попросили «пересобери деку лекции» — скилл должен открыться сам; попросили «поправь опечатку в заголовке слайда» — открыться не должен.

Выберите строку, которая пройдёт обе.

«Собирает презентацию»

«Собирает деку из `deck.yaml` и `slides`»

«...и используй, когда просят пересобирать деку»

«...но не для правки одного слайда»

«дека, слайды, презентация, `rptx`, рендер»

Одна редакция из пяти проходит обе проверки

РЕДАКЦИЯ ОПИСАНИЯ	ГДЕ ОНА РАБОТАЕТ — КОНКРЕТНЫЕ СЛУЧАИ	ОГРАНИЧЕНИЯ
«Собирает презентацию»	там, где «презентация» и есть рабочее слово: им просят, им называют результат	слов «дека» и «слайды» в строке нет, а просят обычно ими — скилл молчит и там, где нужен. До второй проверки дело не доходит
«Собирает деку из <code>deck.yaml</code> и <code>slides</code> »	когда предмет назван словами запроса: на «пересобери деку» скилл откроется уверенно	«поправь опечатку в слайде» — тоже про слайды: откроется там, где не нужен, и будет открываться в каждой второй сессии
«...и используй, когда просят пересобратить деку»	когда назван не только предмет, но и повод: ловит и «пересобери», и «перекатай деку заново»	запрет не сказан — значит не действует. Лишнее срабатывание на правке одного слайда остаётся
«...но не для правки одного слайда»	когда назван повод и отсечён соседний случай: правка одного слайда	описание держит только то, что в нём написано: про соседний скилл с похожим смыслом оно не знает
«дека, слайды, презентация, <code>rptx</code> , рендер»	как подсказка человеку, который ищет файл глазами в чужом репозитории	выбор идёт по смыслу написанного; совпадение слов его не решает. Без глагола не сказано, что скилл делает , — официальный пример «как не надо»

Из сочетания колонок следует состав, и состав один: что делает · когда звать · когда не звать. Третья редакция законна — в проекте, где спутать скилл не с чем, её хватает. Четвёртую берём потому, что рядом лежат соседние скиллы по слайдам, и отсечь их должен сам текст описания. Длина при этом ни о чём не говорит: третья длиннее целевой и всё равно промахивается.

Описание пишут в третьем лице: «собирает деку». Первым лицом — «я помогу собрать» — не пишут.

Два наивных фронтматтера — и три вопроса к третьему

ВАРИАНТ

ЧТО ПРИ ЭТОМ ПРОИСХОДИТ

фронтматтера нет

описанием становится заголовок `# deploy` — дубль имени. По смыслу скилл не откроется никогда. Прямой `/deploy` при этом работает, поэтому автор ничего не замечает

`description:`
Помогает с деплоем

поле не пустое, фронтматтер годный — но не сказано **когда** звать и **когда не**. Это запрещённая документацией формулировка, переведённая дословно

третий — собираем
сейчас

чем он обязан отличаться от второго, называет зал: вопросы ниже

ВОПРОС

КУДА ЛОЖИТСЯ ОТВЕТ

Что этот скилл делает?

первое предложение, третьим лицом

По каким словам его будут звать?

второе, после «используй, когда»

Где он сработать **не** должен?

явное «не для...» в конце

Что продиктовал зал

.claude/skills/deploy/SKILL.md — собрано из трёх ответов

name: deploy

description: Деплоит лендинг signup-landing на прод — сборка, smoke-тест формы, публикация, проверка прод-URL. Используй, когда пользователь просит задеплоить, выкатить или опубликовать сайт — не для обычных правок кода без деплоя.

Проверяется в обе стороны, и вторую проверку почти всегда пропускают. Просим «выкати сайт» — скилл должен открыться. Просим «поправь опечатку в вёрстке» — открыться не должен. Если открылся, описание слишком широкое, и вы купили себе лишнее срабатывание в каждой второй сессии.

Тело: годный текст, в котором нет команд

наивное тело — годный текст, скилл сработает:

1. Собрать проект.
2. Прогнать тесты.
3. Опубликовать.
4. Проверить, что всё работает.

Текст годный, скилл сработает. Не сказано одного: какой именно командой. Модель угадает заново или возьмёт устаревшую — задача угадывания не решена — она переехала из файла инструкций внутрь скилла, где её труднее заметить.

собираем: назовите команду на каждый шаг

1. Собрать: ?
2. Проверить форму: ?
3. Опубликовать: ?
4. Открыть адрес на проде и отправить форму руками
5. Записать, что выложено, когда и какой командой

Тело, которое продиктовал зал

1. Собрать: `npm run build` — код выхода должен быть 0.
2. Проверить форму: `npx playwright test` — должно быть зелёным.
3. Опубликовать: `wrangler pages deploy dist --project-name signup-landing`.
4. Открыть адрес на проде, вручную отправить форму с верными данными — заявка должна прийти.
5. Записать в `.tasks/active/ГГГГ-ММ-ДД-описание.md`: что выложено, когда и какой командой.

КОМАНДА	ЧТО ОНА ДЕЛАЕТ
<code>npm run build</code>	запускает сборку, описанную в самом проекте: собирает исходники в готовые файлы сайта и кладёт их в каталог <code>dist</code> . Код выхода 0 означает, что сборка дошла до конца
<code>npx playwright test</code>	открывает сайт в настоящем браузере и проходит форму так, как её прошёл бы человек. «Зелёным» называют прогон, в котором ни одна проверка не упала
<code>wrangler pages deploy dist -- project-name signup- landing</code>	выкладывает содержимое каталога <code>dist</code> на хостинг. Имя проекта в конце указывает, куда именно выкладывать: без него команда опубликует не туда

Официальное правило с другой стороны: не предлагать несколько способов без необходимости. Ровно один — тот, что в этом репозитории действительно работает. Шагов пять, литеральных команд три: четвёртый и пятый исполняются руками, и выдуманной команды у них нет.

Сфокусированный обгоняет исчерпывающий

ИНТУИЦИЯ, КОТОРАЯ КАЖЕТСЯ БЕЗОБИДНОЙ

«раз уж пишу скилл — напишу его исчерпывающим, пусть покрывает все случаи»

«чем больше в теле написано, тем надёжнее сработает»

ЧТО ИЗМЕРЕНО ПРОТИВ НЕЁ

сфокусированный скилл обгоняет более крупные и исчерпывающие наборы. База: та же выборка из 87 задач, менялась только структура скилла

откроется файл или нет, решает описание; объём тела на это не влияет — до срабатывания тела в контексте нет. Раздутое тело удорожает каждое срабатывание и не улучшает ни одного выбора

Официальная рекомендация с той же стороны: держать файл скилла до 500 строк, а подробный справочный материал уносить в отдельные файлы — те самые вложения, которые стоят ноль, пока не открыты.

Честно про силу утверждения: направление подтверждено первоисточником, а числовой размер разрыва из работы не извлечён. Процента здесь нет, и выдумывать его мы не будем — сказано направление и названа база.

Докуда судьбу скилла решает описание

ОПИСАНИЯ ХВАТАЕТ

скиллов единицы, области у них не пересекаются: спутать не с чем

зовут его теми же словами, что стоят в описании

решение «открывать или нет» вы готовы отдать модели

ЗА ПРЕДЕЛОМ — И ЧТО БЕРУТ ВМЕСТО

у соседа описание похоже → текстом это уже не чинится. Точное совпадение имён разрешает приоритет уровней, похожий смысл не разрешает ничто; это следующий разговор

зовут другими → дописать настоящие слова запроса; удлинять объяснение бесполезно — перечень обрезает **конец** на 1536 знаках

не готовы → флаг `disable-model-invocation: true`: скилл остаётся вызываемым по имени, а модель его описание не видит вовсе

- Назовите скилл, который вы бы модели не доверили выбирать. Третья строка — про него.
- Всё это — про скилл, который вы пишете сами. А его, скорее всего, уже написали до вас.

6

Решаем, что делать, когда задача стоит сегодня, а своё решение — это недели

Задача стоит сегодня, готовое чужое решение на неё уже есть, а своё — это недели работы. Решаем, что в этом случае делать.

1 ОФИЦИАЛЬНЫЙ СКИЛЛ · 4 ЗАМЕРА · 3 УСЛОВИЯ ОТКАЗА

Хук

Скилл

Своё за недели или готовое сегодня

Одна задача, два пути — и по трём меркам из четырёх готовое честно выигрывает

Написать своё	Взять готовое
срок недели	срок минута
качество разбора таблиц хуже	качество разбора таблиц лучше
поддержка при смене формата ваша навсегда	поддержка при смене формата чужая, пока проект жив
что внутри знаете: сами писали	что внутри НЕ ЗНАЕТЕ

Выбор никогда не звучит как «безопасно или опасно» — он звучит как «сегодня или через месяц».

Три мерки из четырёх за готовое. Четвёртая — «что внутри», и до установки ответа на неё нет.

«Недели» — оценка объёма работы, а не измерение.

Задача. Разобрать PDF заказчика: текст, таблицы, заполненные формы. Официальный скилл ровно на это уже есть — документация приводит его описание как образец того, каким описание должно быть.

Момент решения — обычный: задача есть сегодня, готовое ставится за минуту, своё писать некогда. Никто не выбирает между «безопасно» и «опасно»: выбирают между «сегодня» и «через месяц».

Приходит каталог с кодом, а разрешение — разовое

БАЗА

Чужой скилл — **каталог**, и текст в нём только часть: рядом лежат вложения и скрипты. Скрипты запускаются через оболочку, **с правами вашей сессии** — теми же, что у вас.

каталог

права сессии

разовое разрешение

ЧТО УПУСКАЮТ, КОГДА СТАВЯТ

Разрешение вы даёте **один раз**, а права остаются. Содержимое может измениться потом: обновится вложение, обновится скрипт — **согласия на это уже не спросят**.

Состав каталога вы видели на карте скиллов. Здесь к нему добавлены две вещи, которых там не было: чьи права достаются скрипту и сколько раз спрашивают согласие.

Берёте или пишете своё?

Готовый скилл делает нужное лучше, чем сделаете вы, и ставится за минуту. Вместе с ним устанавливается каталог с кодом, который исполнится с правами вашей сессии. Задача при этом стоит сегодня.

Выберите, что делаете.

ВЗЯТЬ как есть

ВЫЧИТАТЬ и ВЗЯТЬ

**ВЗЯТЬ С
ВЫКЛЮЧЕННЫМ
АВТОВЫЗОВОМ**

написать своё

не ставить вовсе

Вычитать то, что исполняется, потом ставить

ВАРИАНТ	ГДЕ ОН РАБОТАЕТ — КОНКРЕТНЫЕ СЛУЧАИ	ОГРАНИЧЕНИЯ
взять как есть	скилл ваш собственный или вычитан лично тем, кому вы доверяете; каталог внутренний, и кладёте в него только вы	«по теме» проверкой не является: из 125 случаев функционального вреда 86 дали скиллы, выглядевшие подходящими задаче
вычитать и взять	предмет общий, и готовое на него уже лежит в каталоге; есть кому до установки прочитать скрипты и вложения; обновления приходят редко	вычитка — снимок на сегодня: про завтрашнее содержимое каталога она молчит. Читают при этом сначала то, что исполняется : скрипты и вложения, на которые ссылается тело
взять с выключенным автовызовом	действие необратимо — выкатка, миграция, удаление; момент вызова вы назначаете сами	закрывает срабатывание не вовремя и молчит про содержимое. Самостоятельным ответом не работает
написать своё	предмет ваш: ваш формат, ваш регламент, ваша область — то, чего в общем каталоге нет	для общей задачи вроде разбора PDF это недели работы и результат хуже; измеренный прирост не достаётся никому
не ставить вовсе	три проверяемых условия: исполняемое некому вычитать ; скрипт обращается наружу — к сети, к ключам; цена ошибки необратима . Хватает одного	платите тем, от чего отказываетесь: прирост измерен. Если ни одного условия нет, отказ стоит вам этого прироста

Из сочетания колонок. Вторая и третья строки друг другу не противоречат: одна про содержимое каталога, вторая про момент вызова. Отсюда порядок: сначала вычитать то, что исполняется, потом ставить; где действие необратимо — с выключенным автовызовом. Пятая верна сама по себе, при своих трёх условиях.

Уносят правило: читают то, что в скилле исполняется. Тело — текст, модель может ему не последовать. Скрипт выполнится обязательно, и с вашими правами.

Под каждым шагом разбора — свой замер

ОБХОД КАЧЕСТВА, 47 150 СКИЛЛОВ

ОТ КАКОЙ БАЗЫ

КАКОЙ ШАГ ПОДТВЕРЖДАЕТ

уже видели: с курированным скиллом решается **50,5%** задач против **33,9%** без него

те же 87 задач, те же 18 связок модели со средой; менялось одно — есть курированный скилл или нет

брать готовое: прирост настоящий и измеренный

качество скилла из каталогов — **6,2 из 12**, у отобранных — **10,1 из 12**

та же работа, та же 12-балльная рубрика: весь корпус против верхнего квартиля

отбирать: прирост дал скилл, прошедший отбор; средний из каталога набирает вдвое меньше

ОБХОД УЯЗВИМОСТЕЙ, 31 132 СКИЛЛА

ОТ КАКОЙ БАЗЫ

КАКОЙ ШАГ ПОДТВЕРЖДАЕТ

26,1% скиллов несут уязвимость, **5,2%** — признаки злого умысла

детектор с точностью 86,7% и полнотой 82,5%; сплошной обход корпуса

вычитывать до установки: после неё читать поздно

скиллы с исполняемым скриптом уязвимы в **2,12** раза чаще

тот же корпус; различие одно — есть внутри скрипт или нет

читать **сначала скрипты**, тело последним

Две таблицы — две разные работы: свой корпус, своя методика, свой вопрос. Складывать их числа нельзя, и совпадения долей ждать неоткуда.

Что читать в чужом скилле и в каком порядке

```
чужой-скилл/  
├─ scripts/*.sh, *.py    ← 1. читать первым: исполнится обязательно  
├─ вложения (*.md, *.json) ← 2. на них ссылается тело, обновят без спроса  
├─ SKILL.md — фронтматтер ← 3. описание решает, когда скилл сработает  
└─ SKILL.md — тело      ← 4. читать последним: это текст
```

ГДЕ	ЧТО ИЩУТ
скрипты	обращения наружу: сеть, переменные окружения, ключи, запись за пределы каталога проекта
вложения	то же самое, плюс ссылки на файлы вне каталога скилла
фронтматтер	слишком широкое описание — оно решает, когда скилл сработает; и нет ли выключенного явного вызова
тело	шаги, которые делают необратимое, — и нет ли среди них запуска чего-то ещё

```
$ claude plugin validate ./chuzhoy-skill      # прочитать каталог и сказать, что в нём сломано  
Validating skill: ./chuzhoy-skill/skills/pdf-razbor/SKILL.md  
✗ Found 1 error:  
  > frontmatter: YAML frontmatter failed to parse: YAML Parse error: Unexpected  
    token. At runtime this skill loads with empty metadata (all frontmatter  
    fields silently dropped).  
✗ Validation failed                          # код возврата 1; при одних предупреждениях — 0
```

Машина берёт на себя одно: находит фронтматтер, который не читается, до установки и без запуска сессии. Остальные три места читает человек глазами. И каталог ей нужен с манифестом плагина — на голом каталоге скилла она отвечает «манифест не найден».

Вычитали и взяли — а потом оно поменялось

Один открытый каталог, два сплошных обхода с разницей в одиннадцать дней

первый обход



341 вредоносных
это 12% каталога

через 11 дней



824 вредоносных
это 8% каталога

Каталог вырос почти вчетверо, вредоносных стало вдвое больше — а доля НЕ выросла: 12% против 8%.

Вычитка — снимок на сегодня. Она ничего не говорит про то, что положат в каталог завтра.

Конвертер картинок — официальный открытый скилл самого производителя. Внутри добавили одну функцию: тихо скачать и запустить программу-шифровальщика. Снаружи он не изменился: то же имя, то же описание, то же видимое поведение. Исследователи, показавшие это, раскрыли находку производителю; ответ был — ответственность за то, какие скиллы вы запускаете, лежит на вас.

Отсюда следует поправка к порядку: обновление чужого скилла — это новая установка. Её и пропускают, приняв обновление за мелочь. Вывод «не берите чужое» из этой истории не следует.

Когда чужой скилл — правильный выбор

БЕРИТЕ ЧУЖОЙ — ЭТО ВЕРНОЕ РЕШЕНИЕ

предмет **общий**: разбор формата, работа с документами, типовое преобразование — писать своё значит платить неделями за худший результат

исполняемое **есть кому вычитать**, и вычитка успевает до установки

действие **обратимо**, а если нет — ставят с выключенным автовызовом

скиллов в проекте немного и области не пересекаются

ЗА ПРЕДЕЛОМ

предмет ваш: ваш регламент, ваш формат, ваша предметная область → своё, и тогда недели оправданы

вычитывать некому → не ставить: это одно из трёх условий отказа, и оно самое частое

необратимое действие плюс автовызов → такое сочетание не устанавливают

их десятки → библиотека начинает мешать сама: на тех же задачах при росте библиотеки до 202 скиллов **падение успешности доходит до 21%**, и падает именно точность выбора

- **Те же три условия — про хук из чужого репозитория.** Хук лежит в репозитории и приезжает с клоном: устанавливать нечего, а выполняется он с вашими правами так же, как скрипт в чужом скилле. Клонировали чужой проект — впустили чужое исполняемое, не приняв ни одного решения.
- **И про другого агента.** Устройство скилла стандартизовано, тот же SKILL.md читают десятки агентов, а перенос скилла — это копирование каталога. Вместе с каталогом переносится и скрипт внутри: три условия слева работают там же и так же.

Тот же барьер на другом агенте

АГЕНТ	ЧТО ДЕЛАЕМ С НАШИМ БАРЬЕРОМ	ЧТО СТАНОВИТСЯ С РЕШЕНИЕМ
Cursor, Codex, Copilot	Cursor читает наш <code>.claude/settings.json</code> сам — правок нет; Codex и Copilot — переложить файл. Тело скрипта не меняется ни у кого	все пять решений на месте: событие, матчер, условие, оба ответа
Gemini CLI	переписать имя события, файл и форму ответа — около шести строк; тело скрипта остаётся	"timeout": 10 переносится безупречно и становится 10 миллисекундами : барьер молча отваливается по сроку
Windsurf	переписать событие и поле входа; ответ — только код выхода	состояния «спросить» нет вовсе: решение «что делать, когда проверка не смогла ответить» выразить нечем
Aider	механизма нет совсем — ни хуков, ни доступа наружу, ни субагентов: трёх из пяти	решение то же, держит его защита ветки на площадке: слой надёжнее исходного

Из сочетания колонок: пять решений, которые мы принимали на проектировании барьера, переехали все до одного. Переписывать пришлось объявление и форму ответа — напечатанное. Решённое переписывать не пришлось. Там, где механизма нет, решение не отменилось: оно переехало на слой, который держит его крепче.

Честный пробел: всё выше проверено по документации поставщиков. Запуском не проверено ничего. У Cursor это острее прочих: «правок нет» выведено из описания его собственного события — какой вход получает перенесённый хук, документация не описывает.

Две строки оси закрыты

Вторая строка заполнена. Те же пять усилений, что в начале занятия, — у двух из них теперь есть замер.

СТУПЕНЬ	ПРЕДОТВРАЩАЕМ / РЕШАЕМ	ЧТО ОБЕЩАЕТ УСТРОЙСТВО	ЧТО ПОКАЗАЛА ПРОВЕРКА
Хук	Решаем — правило уже нарушено	«Поставили хук — коммит в main закрыт»	Из шести форм одной команды отклоняет одну, на пяти молчит; человека с терминалом не видит
Скилл	Решаем — по сигналу перегрузки файла инструкций	«Важно содержание; описание — деталь, допишем позже»	Без точного описания-триггера не сработает вообще: выбор идёт по описанию, а тела файла до срабатывания в контексте нет
МСР			
Субагент			
Процесс			

Переносится решение, а не запись. Что считать нарушением, когда вмешиваться, что вынести отдельным файлом, какие права урезать — это ваше, оно не принадлежит инструменту. Имя события, папка, форма ответа «нет», единицы срока — у каждого агента свои. А механизма может не быть вовсе: решение тогда в силе, держать его приходится другим уровнем, часто более надёжным.

«Две строки заполнены. Три остаются на следующее занятие.»

Что появилось в репозитории

«На входе в занятие каталога `.claude/` в репозитории не было.»

github.com/tellina-study/signup-landing-demo ветка seminar-5-hook, коммит 9803643

<code>.claude/</code>	← на входе не существовал
<code>settings.json</code>	← хук · подтверждено коммитом
<code>hooks/selftest-branch-guard.sh</code>	← хук · подтверждено коммитом
<code>skills/deploy/SKILL.md</code>	← скилл · только в плане, коммита нет

CLAUDE.md 24 строки / 4 раздела	→	40 строк / 5 разделов	← хук · подтверждено коммитом
+ ## Хук защиты ветки			← барьер и кого он НЕ останавливает
+ 3-й пункт в Repository etiquette			← он написан уже под хук

Два файла подтверждены коммитом, один существует только в плане: запись о будущей работе не равна выполненной.

Тот же вопрос — второй раз

Этот вопрос уже звучал в начале занятия; ответ тогда не озвучивался.

В этом репозитории — с его формой заявки, тестами и двадцатью четырьмя строками инструкций — какое из пяти вы завели бы первым? У кого ответ изменился — и что его изменило?

хук

скилл

МСР

субагент

процесс

Что за час стало проверяемым

Под какими строками лежит файл, который можно открыть

хук	барьер стоит и сам печатает свои дыры	settings.json + самотест ветка seminar-5-hook, 9803643
скилл	судьбу решает описание, а не содержимое	описание-триггер во фронтматтере
МСП		
субагент		
процесс		

Под двумя карточками из пяти теперь лежит файл: его можно открыть и проверить командой. Под тремя остальными такого файла нет.

«Три нижние строки сегодня ничем не проверены: про них сказано только то, что их ступени впереди. Выдавать это за результат нет оснований.»